

Building AI Tutors with KIVA

MA AI Hub x SundAI — Reimagining Education with AI

Hack Day Pre-Session

MIT AI Hub

May 6, 2026

Outline

- 1 Event Overview
- 2 What is KIVA?
- 3 System Architecture
- 4 Installation Guide
- 5 Activity Structure
- 6 Learning Verification Without Data Storage
- 7 Hack Day Opportunities
- 8 Resources & Next Steps

Mission

Explore how AI can reshape learning through hands-on prototyping

Schedule

- **Pre-session:** May 6, 6:00-7:30 PM
- **Hack Day:** May 9, 9:00 AM-5:00 PM

Who Should Join

- Educators
- Researchers
- Developers & AI builders
- Students

① AI Tutors — Build on KIVA

- Evaluate tutor behavior in real scenarios
- Prototype improvements: feedback flows, personalization, safety
- Analyze educator interaction and intervention points

② Learning Path Engines

- Generate personalized learning pathways
- Sequence real content (papers, courses)
- Adapt to learner background

③ Future of Education Systems

- Build educator tools
- Create new learning experiences
- Push education forward

Definition

KIVA is an **open-source AI tutoring platform** built on Riverst that enables interactive, speech-driven learning experiences with AI avatars.

Key Features

-  Real-time avatar interaction
-  Speech-based conversations
-  Automated analysis
-  Customizable flows
-  Session recording

Use Cases

- Vocabulary tutoring
- Language learning (ESL/ISL)
- Audiobook comprehension
- Custom educational activities

- **Riverst Platform Demo**

- Overview of capabilities
- <https://drive.google.com/file/d/1r2LoBGbjBx1mdDIv-fPzGeZVEYf1kLw8/view>

- **KIVA Vocabulary Activity**

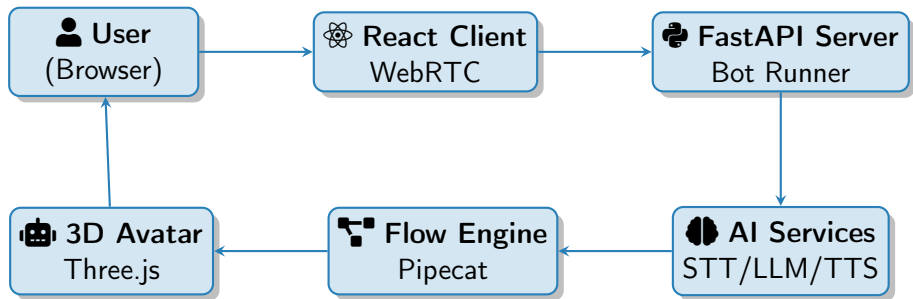
- Vocabulary teaching workflow demonstration
- <https://drive.google.com/file/d/1jTHvTXG4WWIYqgqjC4lp1LWMEnyZJQzr/view>

- **Activity Summary & Configuration**

- Configuration pages and automated analysis
- https://drive.google.com/file/d/1AvYnSq87ne0Yj_YSduYatN5D4rd0-7QG/view

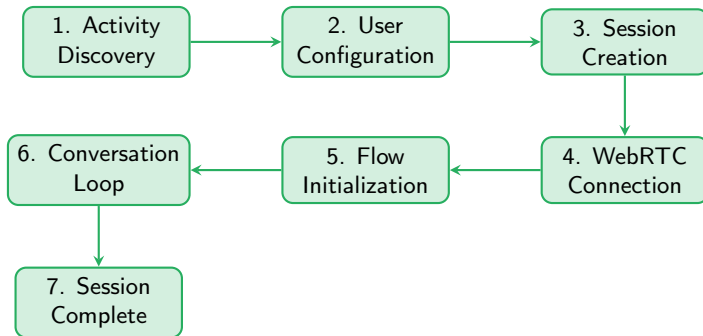
 Watch these before the hack day!

How KIVA Works: High-Level Architecture



Real-time Pipeline: Speech → STT → LLM → TTS → Avatar Animation

Activity Lifecycle



- Frontend loads activities from `/api/activities`
- User configures via session schema
- Backend establishes WebRTC and initializes AI pipeline
- Flow system manages conversation state and transitions

Core Components

Frontend (React)

- `Homepage.tsx` — Activity browser
- `AvatarInteractionSettings.tsx` — Config UI
- `AvatarRenderer.tsx` — 3D avatar (Three.js)
- WebRTC connection management

Backend (FastAPI)

- `main.py` — API endpoints
- `bot_runner.py` — Orchestration
- `flow_factory.py` — Flow builder
- `handlers.py` — State management

AI Services

- **STT**: OpenAI Whisper / Local Whisper
- **LLM**: OpenAI GPT / Ollama (local)
- **TTS**: OpenAI TTS

Flow System (Pipecat)

- Stage progression tracking
- Function calling system
- Transition logic
- Context management

Analysis (Senselab)

- Speech feature extraction

System Requirements

Supported Platforms

- ✓ Apple Silicon (M-chip) macOS
- ✓ Ubuntu Linux
- ✗ Windows (not supported)

Software Requirements

- Python 3.11+
- Node.js 16+
- FFmpeg 7.*
- Modern browser (Chrome 134+)

API Keys (Optional)

- OpenAI API key
- Hugging Face token
- Google OAuth (optional)

Note

Not all API keys are required. Local models (Ollama, Whisper) can run without external APIs.

Installation: Step 1 — Clone Repository

```
# Clone the repository
git clone https://github.com/sensein/riverst.git
cd riverst
```

Repository Structure

riverst/ contains src/server/ (FastAPI backend), src/client/react/ (React frontend), KIVA/ (documentation), and docker-compose.yaml

Installation: Step 2 — Environment Setup

Server Configuration

- `cd src/server`
- `cp env.example .env`
- Edit `.env` and add your API keys

Client Configuration

- `cd src/client/react`
- `cp env.example .env`

Minimal `.env` for Server

```
OPENAI_API_KEY, HF_TOKEN, ANALYZE_AUDIO=true
```

Installation: Step 3 — Install Dependencies

Option A: Using Conda (Recommended)

- `conda create -n riverst python=3.11`
- `conda activate riverst`
- `conda install -c conda-forge ffmpeg`
- `pip install -r requirements.txt`

Option B: Using Docker

- `docker compose up -build`

Client setup

- `cd src/client/react && npm install`

Installation: Step 4 — Run KIVA

Start Server (Terminal 1)

- `cd src/server`
- `conda activate riverst`
- `python main.py`
- Server runs on `http://localhost:7860`

Start Client (Terminal 2)

- `cd src/client/react`
- `npm run dev`
- Client runs on `http://localhost:5173`

Important

⚠ The server **must** be running before starting the client!

Quick Start Scripts

Verification

- 1 Open browser to `http://localhost:5173`
- 2 You should see the Riverst homepage
- 3 Look for "**Knowledge Integration and Vocabulary Acquisition through AI (KIVA)**" section
- 4 Try the "**Child vocabulary training**" activity

Troubleshooting

- **Port conflicts:** Check if 7860 or 5173 are in use
- **API errors:** Verify `.env` has valid OpenAI key
- **FFmpeg missing:** Install via conda or system package manager
- **Browser issues:** Use Chrome 134+ for WebRTC support

Simple Activities

- Only `session_config.json`
- Open-ended conversations
- No structured flow
- Example: `basic-avatar-demo`

Advanced Flow Activities

- `session_config.json` + `flow_config.json`
- Structured conversation stages
- Dynamic content adaptation
- Example: `vocab-tutoring`

For Hack Day

You'll primarily work with **advanced flow activities** to build sophisticated tutoring behaviors.

Session Configuration

`session_config.json` defines activity parameters and UI options:

Key Configuration Fields

- AI service selection (STT/LLM/TTS)
- Avatar behavior prompts
- UI options (video, transcripts)
- Embodiment type (avatar vs waveform)

Key Fields:

- AI service selection (STT/LLM/TTS)
- Avatar behavior prompts
- UI options (video, transcripts)

Flow Configuration

`flow_config.json` defines conversation structure:

Flow Elements

- **Nodes:** Conversation stages (greeting, teaching, review, end)
- **Transitions:** Movement between stages based on conditions
- **Actions:** Custom Python handlers (pre/post)
- **Role messages:** System prompts for each stage

Flow Elements:

- **Nodes:** Conversation stages
- **Transitions:** Movement between stages
- **Actions:** Custom Python handlers (pre/post)

How Does KIVA Know Learning Happens?

Privacy-First Approach

KIVA can verify learning **without storing any personal data**

What KIVA Does NOT Store:

- No user profiles
- No learning history
- No persistent data
- No cross-session tracking

What KIVA CAN Verify:

- In-session progress
- Real-time comprehension
- Immediate application
- Session-end mastery

Strategy 1: Contextual Memory (LLM)

The LLM remembers the entire conversation in its context window:

[0:00] Avatar: "Today we learn 'apple' = manzana"

[0:05] User: "What does apple mean?"

[0:05] Avatar: "Apple means manzana!"

[0:10] User: "Manzana!"

[0:25] Avatar: "Remember 'apple'? → User: "Manzana!" ✓

Strategy 2: Session State Tracking

Temporary state maintained only during active session:

Session State (Ephemeral)

```
{  
  "words_introduced": 5,  
  "words_correct": 4,  
  "words_failed": 1,  
  "current_difficulty": 2  
}
```

Enables Real-Time:

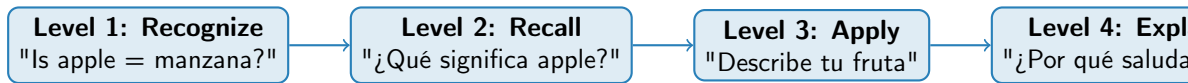
- Progress tracking
- Difficulty adaptation
- Spaced repetition
- Mastery detection

Key Point

State is **deleted** when session ends — no persistence

Strategy 3: Progressive Verification

Bloom's Taxonomy applied within a single session:

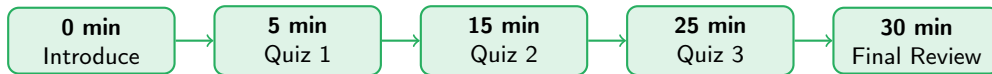


How It Works

Each level builds on previous — if student fails Level 3, avatar knows they passed Level 2 but need more practice

Strategy 4: Spaced Repetition (In-Session)

Testing at increasing intervals within same session:



If Correct:

- Increase interval
- Move to next concept

If Incorrect:

- Provide remediation
- Re-test sooner

Strategy 5: Real-Time Pattern Detection

Analyzing user behavior during conversation:

Behavioral Signals

- **Hesitation** — Long pauses before answering
- **Self-correction** — "Wait, I mean..."
- **Confidence** — Voice tone, speed
- **Accuracy** — Correct vs incorrect responses

Adaptive Response

- Slow down if hesitation
- Provide hints if struggling
- Challenge if confident
- Celebrate if accurate

Example

If user pauses 5+ seconds → Avatar: "Take your time, here's a hint..."

Session Summary (No Storage Required)

At session end, KIVA generates a summary:

Example Summary

"Today's session covered:

- ✓ 5 vocabulary words introduced
- ✓ 4 mastered on first try
- ✓ 1 needed 3 attempts (orange)

Recommendation: Review 'orange' in next session"

User Options:

- Download summary as PDF
- Email to themselves
- Copy to notes
- Share with teacher

Privacy Benefits:

- User controls their data
- Nothing stored on server
- GDPR compliant
- No consent needed

Trade-offs: No Persistent Storage

✓ Advantages

- Complete privacy
- GDPR compliant by default
- No data breaches possible
- Simpler infrastructure
- Works offline
- No consent management

✗ Limitations

- No long-term progress tracking
- Each session starts fresh
- No personalized curriculum
- No cross-session adaptation
- User must track own progress

Hybrid Approach (Optional)

Allow users to **opt-in** to long-term memory for personalized learning

What You Can Build

Evaluation & Analysis

- Analyze tutor behavior in real scenarios
- Study conversation logs and patterns
- Identify where the AI breaks down
- Determine when human intervention is needed

Prototype Improvements

- **Feedback flows:** Better assessment and correction
- **Personalization:** Adapt to learner level and style
- **Intervention checkpoints:** When to alert educators
- **Analytics dashboards:** Track progress and engagement
- **Safety layers:** Content filtering and guardrails
- **Educator controls:** Tools for teachers to guide sessions

Example Projects

① Adaptive Difficulty System

- Monitor student performance in real-time
- Adjust vocabulary complexity dynamically
- Implement spaced repetition

② Educator Dashboard

- Live session monitoring
- Intervention triggers (student struggling)
- Post-session analytics and insights

③ Multi-Modal Assessment

- Analyze speech patterns (pronunciation, fluency)
- Track engagement via video (attention, emotion)
- Combine metrics for holistic evaluation

④ Safety & Content Filtering

- Detect inappropriate content
- Age-appropriate language adaptation
- Bias detection and mitigation

Technical Extension Points

Backend Extensions

- Custom handlers in `handlers.py`
- New flow configurations
- Additional AI service integrations
- Database for learner profiles
- Real-time analytics pipeline

Frontend Extensions

- New activity configuration UIs
- Live session monitoring
- Educator dashboards
- Student progress views

Flow System Extensions

- Branching logic based on performance
- Dynamic content loading
- Multi-stage assessments
- Personalization algorithms

Analysis Extensions

- Custom Senselab pipelines
- Speech quality metrics
- Engagement detection
- Learning outcome prediction

Documentation & Resources

- **GitHub Repository:** <https://github.com/sensein/riverst>
- **Activity Creation Guide:** `src/server/activities/ACTIVITY_CREATION_GUIDE.md`
- **Project Board:** <https://github.com/orgs/sensein/projects/55>
- **Pipecat Framework:** <https://github.com/pipecat-ai/pipecat>
- **Senselab:** <https://github.com/sensein/senselab>

Before Hack Day

- 1 Install KIVA and verify it runs
- 2 Watch the demo videos
- 3 Explore the vocab-tutoring activity
- 4 Read the Activity Creation Guide
- 5 Think about what you want to build!

During Pre-Session (May 6)

- Installation troubleshooting
- Architecture questions
- Project ideation
- Team formation

During Hack Day (May 9)

- Technical support
- Code reviews
- Integration help
- Demo preparation

Communication Channels

- Event Slack/Discord (TBD)
- GitHub Issues for bugs
- In-person mentors on hack day

Quick Reference: Key Commands

Installation

```
git clone https://github.com/sensein/riverst.git
conda create -n riverst python=3.11
conda activate riverst
pip install -r requirements.txt
```

Running

Terminal 1: `python main.py` (in `src/server`)
Terminal 2: `npm run dev` (in `src/client/react`)

Access

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:7860`

Questions?

Let's build the future of education together!

 <https://github.com/sensein/riverst>

See you on May 9!